Study Guide: Everything We Built on September 19, 2026

For Steve Powell. Written just after midnight (September 20) from the working session, so it reflects what actually happened, including the mistakes. Everything named here is in F:\ClaudeProjects unless it says otherwise.

Contents

1. The day at a glance

2. Email Downloader: from a download tool to a threat-triage tool

3. The PowellTec Support Tool, ported to pure PowerShell (PsSupportTool)

4. Worship Slides: the rebuild with hymn search

5. Church Kids: the new children's lesson maker

6. Bordeom: the fun collection

7. Where everything lives

8. Concepts worth studying (with where to see them)

9. Command cheat sheet

10. Mistakes we made and what they teach

11. Open items and what to do next

12. A suggested way to study all this

1. The day at a glance

One long stretch, five bodies of work (the PowellTec port ran from the evening of September 18 to about 5 AM on the 19th). Roughly in order of size:

Project	What happened today	Where it lives	Status
PowellTec Support Tool port (PsSupportTool)	Your large .NET support-tool app (MAUI, about 15,000 lines) was ported to a pure PowerShell + WinForms version for your new job, which does not allow compiled executables: 148 tool functions in 9 modules, a window that builds itself from the functions (section 3).	D:\Build Projects\Old PowerShell\PsSupportTool\	v1 works; not in git yet
Email Downloader	Went from 'save some emails to text files' to a working phishing-triage tool: Outlook made to work live in all three languages, a ranked risk report, abuse-report drafts, a single-file exe for your boss,	email_downloader\ (git repo)	Working, committed

	and a pure-PowerShell version with its own window for use at work (section 2.10).		
Worship Slides (the church parser)	Found that no build had the hymn feature; archived the July builds and rebuilt both exes. You used it on the Sept 20 packet.	AgentBuilds\Church\ (dist\ has the exes)	Working, used in real life
Church Kids (new)	A children's version: Bible stories with pictures, Bible verses at four reading levels, Word documents, drawn themes, credits slide, exe. Your test deck: 40 slides, 'almost perfect'.	ChurchKids\ (dist\ChurchKids.exe)	Phase 1 done
Bordeom (new)	Four nostalgic programs: an Acidwarp tribute, a bouncing line, a Mandelbrot zoomer, and a Morse code tool for your father.	Bordeom\ (dist\MorseCode.exe)	Working

Timeline (roughly)

- Evening of Sept 18 to about 5 AM Sept 19: the PowellTec Support Tool ported to pure PowerShell (section 3).
- Morning: recovered context after the crash, found the Outlook sign-in blocked by the Azure app registration.
- Afternoon: got the redirect address and sign-in authority right, and Outlook worked live. Added Junk-folder search (in:spam), the risk report, and per-sender abuse drafts.
- Evening: live-tested all three languages on 99 real Junk messages (identical output), explained the Gmail from:me quirk, built the exe and the clean zip for your boss.
- Night: Bordeom (four programs), the worship-slides rebuild, then Church Kids from idea to tested exe.
- After midnight: repaired the corrupted credentials.json in the dist folder, and this guide.

2. Email Downloader: from a download tool to a threat-triage tool

2.1 What it is now

One program design, three complete versions that behave identically: Python (email_core.py plus provider files), PowerShell (EmailDownloader.ps1, one script with its own window: see section 2.10), and C# (a WinForms app). SPEC.md is the contract all three follow, and testdata\ holds shared test cases whose expected outputs must match byte for byte in every language. That is the key idea to study: write the rules once, then prove three implementations agree.

It reads Gmail or Outlook/Hotmail, saves the newest N matching messages as text files, and can also save an HTML copy with pictures. Two modes turn it from a downloader into a triage tool:

- **Inspect mode:** adds a section to each saved message listing the headers (From vs Reply-To, Return-Path, SPF/DKIM/DMARC), every attachment (never opened) and every link, each shown 'defanged' (hxxps://evil[.]example) and raw, with warning flags. The saved HTML has links turned into inert text.
- **Report mode:** after a run, writes a ranked summary (.txt) and a spreadsheet-style file (.csv), plus draft abuse reports you can send yourself. Nothing is sent, opened, or looked up for you.

2.2 The warning flags (Inspect mode)

Flag	Meaning	Score weight
AUTH-FAIL	SPF, DKIM or DMARC failed in the delivery headers	3
CLOUD-HOSTED	A link goes to a public hosting service (Google Cloud Storage, S3, web.app...)	3
JAVASCRIPT / USERINFO / IP-HOST / PUNYCODE	javascript: links; user:pass@host tricks; numeric-IP links; look-alike (xn--) domains	3 each
STYLED-TEXT	Fancy Unicode letters (like bold math letters) in the From or Subject, used to dodge filters	2
DIFFERS FROM FROM	Reply-To address differs from From (or the display name hides the address)	2
RISKY TYPE	Attachment types like .docm, .exe, .js	2
MISMATCH	Link text says one site but the link goes to another (very common in harmless marketing mail)	1
PIXEL, TEXT-ONLY, REDIRECT	Information only, no score: 1x1 tracking image; a website named in the text that no link goes to; another address embedded in a link	0

Score = the sum of the weights of each flag present (not multiplied by how many times). Messages are ranked by score. In practice: harmless bulk mail (Guns.com, Meijer, Tropical Smoothie) scored 1 to 3; Junk-folder spam scored up to 5.

2.3 Getting Outlook to work (the hard part, and why)

A program that reads someone's mail must be registered with the mail provider once. Google gives you credentials.json; Microsoft needs an 'app registration' in Azure. The path to make it work today, in order:

- First problem: portal.azure.com would not create an app registration for a personal Microsoft account ('Selected user account does not exist in tenant Microsoft Services'). The earlier diagnosis (that one account was 'tangled') was wrong: this is how personal accounts behave, because the portal connects them to a placeholder tenant with no directory.
- You then got past that wall (the session that crashed noted it saved the card-and-subscription route; I do not know your exact clicks) and registered an app called 'EmailChecker' under powellforwork@outlook.com, with supported account type = personal Microsoft accounts only.
- Redirect address: sign-in returns the browser to http://localhost. The portal form said 'must be https'; editing the Manifest silently reset. What worked: Authentication, Add a platform, Mobile and desktop applications, custom redirect http://localhost, then Allow public client flows = Yes.
- Code fix 1: the sign-in endpoint. An app limited to personal accounts must use login.microsoftonline.com/consumers, not /common (Microsoft answered with a clear 'userAudience' error).
- Code fix 2: the msal library had to be installed (pip install msal) for Python.
- Code fix 3: always show the account chooser (prompt=select_account), otherwise the browser silently picks whichever Microsoft account is signed in (it kept picking the empty test account).

Result: your real Hotmail account (caeled@hotmail.com) signs in and downloads live, in Python, PowerShell and C#.

2.4 Folder search (Gmail-style, for Outlook)

Typing exactly in:spam (or in:junk, in:inbox, in:sent, in:trash) as the search reads that Outlook folder, newest first, using the Graph endpoint /me/mailFolders/junkemail/messages. Any other text is a normal keyword search across all folders. It is written up in SPEC section 3.2.

2.5 Live results

- Gmail, spam folder, 50 messages: 16 had a risk flag, 34 none. The Google-Cloud phishing campaign grouped into 6 hosting buckets, with abuse drafts.
- Outlook, Hotmail Junk, 99 messages: 58 flagged, 41 clean. No cloud-hosted links, so no hosting drafts.
- Same 99 messages in all three versions: all 198 message files, the report and the CSV were byte-for-byte identical between Python, PowerShell and C#.
- Your inbox bulk mail (Guns.com, Meijer, Tropical Smoothie, about 50 messages) never scored above 3, so a cut-off near 4 separates it from Junk in this data.

2.6 Sender abuse drafts (new today)

Spam that links to no cloud host still has a sender. Messages whose SPF actually failed are grouped by the sending IP address (the first public IPv4 in Received-SPF, then Authentication-Results; private ranges skipped). Each group gets a draft 'mail from your network failed sender authentication' with the header evidence, up to three examples. The tool does not look up who owns the IP: you do that with a whois service, then send it yourself. On your Junk sample: 8 sender IPs covering 13 messages (I said 14 in chat; 13 is the correct count).

The false positive that shaped the rule

A legitimate Google Groups mailing-list message (SPF pass, DKIM pass, only DMARC fail) produced a draft aimed at Google. Mailing lists and forwarders re-send your mail from their own servers, so DMARC alignment fails even though nothing is wrong. Fix: a draft needs SPF itself to have failed. The message still shows AUTH-FAIL in the ranking. Lesson: test on real, ordinary mail, not just on spam.

2.7 The Gmail 'from:me' oddity, solved

With 'only emails I sent to myself' ticked, Gmail's from:me to:me let strangers through in the spam folder. The tool only passes the query along, so it is Gmail's behaviour: Gmail matches each dot-separated piece of a sender's domain as a search word (from:agniatreo finds mail from x.agniatreo.me), so from:me also matches every .me domain (and me.com). Decision: leave it and document it (SPEC section 3.1).

2.8 The exe for your boss

```
dotnet publish csharp\EmailDownloader -c Release -r win-x64 --self-contained -
p:PublishSingleFile=true -p:EnableCompressionInSingleFile=true -o dist\EmailDownloader
```

- Result: dist\EmailDownloader\EmailDownloader.exe (about 50 MB, includes .NET, no install).
- Send only the exe plus azure_config.json (a zip of exactly those two is on your Desktop: EmailDownloader_for_boss.zip). The client ID in azure_config.json is not a secret; it lets any personal Outlook/Hotmail/Live account sign in.

- Never send the dist folder: it holds your own credentials.json, token_store and token_cache_cs.bin. Gmail users would need credentials.json and to be added as test users in your Google Cloud consent screen.
- The exe is unsigned, so Windows shows an unknown-publisher warning (More info, Run anyway).
- *.exe and *.zip are in .gitignore, so builds never enter the repo; the README records the build command instead.

2.9 The credentials.json incident (after midnight)

Your copy in dist\EmailDownloader had shrunk to 54 bytes and was not valid JSON (changed 10:55 PM). Six other copies were identical (413 bytes, valid, Google 'installed' format). Fix: renamed the broken file to credentials.json.broken-2026-09-19-2255 and copied a good one over it, checking sizes and fingerprints only, never displaying the contents. Lesson: keep known-good copies of small critical files, and compare by hash.

2.10 The pure PowerShell version, with its own window (for your current job)

You asked for a version that is nothing but PowerShell, so it can be used where you work: one plain script, nothing to install and nothing to compile, that you can read line by line. That is powershell\EmailDownloader.ps1: about 1,200 lines, Windows PowerShell 5.1, with a WinForms window like the C# one. It does everything the other two do (Gmail, Outlook, the Junk folder search, HTML export, Inspect mode, the report and the abuse drafts).

Fact	Detail
Open the window	.\EmailDownloader.ps1 (or: powershell -File powershell\EmailDownloader.ps1)
Command line	.\EmailDownloader.ps1 -Cli -Provider outlook -Query "in:spam" -Count 50 -Report -Out C:\temp (other switches: -SelfOnly -Html -Inspect)
Self-test	.\EmailDownloader.ps1 -SelfTest (it must end with ALL PASSED)
Files that sit next to the script	Gmail: credentials.json. Outlook: azure_config.json holding the client ID. The first run opens a browser to sign in; tokens are cached in token_ps_gmail.json and token_ps_outlook.json (its own files, not shared with the Python or C# versions). Keep all of these off shared drives; .gitignore already keeps them out of git.
Why the file is ASCII only	Windows PowerShell 5.1 reads a script that has no byte-order mark as the old ANSI code page, so any fancy character in the file would be misread. Special characters are built in code, for example [char]0x2026 for the ellipsis.
How the window stays responsive	The download runs in a background runspace (a separate PowerShell thread). The script loads its own functions there with the -Library switch.
What it needs	Only Windows PowerShell 5.1 and .NET's WinForms, which come with Windows. TLS 1.2 is switched on in the script.

What was proven today, and the bugs found in this version

- Live-tested with real mail: Gmail sign-in, the window, the command line and -Report; then Outlook. On the same 99 Junk messages its output (message files, report and CSV) was byte-for-byte identical to Python's and C#'s.
- **Bug 1, the window's layout:** rows were drawn on top of the checkboxes (the '# of emails' box was hidden). Cause: one part of the code used a local variable $y while the row helper advanced $script:y. Fix: everything in Show-Gui uses $script:y. How it was verified: the form was rendered to a PNG before and after (swap ShowDialog() for Show() and use DrawToBitmap).

- **Bug 2, the window ignored the count and the 'sent to myself' box:** the background runspace's parameters had the same names as the script's own parameters (Count, SelfOnly); when the script loaded, the script's defaults overwrote the window's values. Fix: the runspace parameters are now $WantCount and $WantSelfOnly. The rule to keep: never name a runspace parameter the same as a script parameter. Your 99-message run through the window confirmed it works.
- **Bug 3, an emoji in HTML:** Windows PowerShell 5.1's WebUtility.HtmlDecode leaves numeric entities above U+FFFF (such as the padlock emoji written as 🔒) undecoded. Fix: a small HtmlDec helper; SPEC section 5 was updated and a test case (gmail_html_nested) was added. Found only because the Python and PowerShell outputs were compared on real mail.
- Also done in this version today: the /consumers sign-in authority and the account chooser for Outlook (-AuthExtra 'prompt=select_account'), the in:spam style folder tokens, and the sender abuse drafts with the SPF rule.

Before you point it at your work mailbox (important)

- **Outlook limit:** the app registration (EmailChecker) accepts personal Microsoft accounts only (Hotmail, Outlook.com, Live), and the script signs in through the /consumers endpoint. A work or school Microsoft 365 mailbox will not sign in as things stand. It would need an app registration that allows organization accounts and a different authority in the script (see python\AZURE_SETUP.md), and some organizations require an administrator to approve the Mail.Read permission.
- **Gmail:** credentials.json works with a Google account only if that account is allowed to use your Google Cloud client (test users on the consent screen; in testing mode the sign-in expires after 7 days).
- **Policy:** check your workplace's rules on downloading mail and on using your own app registration. That is your call to make, not something a script should decide.
- **Execution policy:** the script is not digitally signed, so a locked-down machine may refuse to run it. If it is blocked, treat that as an IT policy question rather than something to work around.

3. The PowellTec Support Tool, ported to pure PowerShell (PsSupportTool)

This is the biggest project in the guide, and the first version of this guide left it out. It was built mostly in the sessions of September 16 to 19 (the last files were written around 5 AM on the 19th). This section is based on the project's own README, my notes from those sessions, the file dates, and a check I ran tonight (PowerShell's own parser read every file). I did not watch it being built in this session and I have not run its tools against any computer tonight.

3.1 What it is, and why there are two versions

- **The original, PowellTec IT Support Tools v2:** a .NET MAUI application (folder SupportToolApp) about 15,000 lines of C# and XAML, built for Windows, Mac, Android and iOS. Around it: an older Avalonia build (ToolsAvaloniaApp), a relay service so a phone can run PowerShell on a computer (PowellTecRelay), and a GitHub repository with an automatic build for four platforms. Started June 27, 2026 as your own tool and a portfolio piece; it grew out of the older PowerShell WinForms tools you wrote at a previous job. It keeps being developed and is NOT replaced by the PowerShell port.

- **The port, PsSupportTool:** a separate, standalone build made for your new job as a Field IT Engineer. The employer allows PowerShell (and SSH, separately approved) but will not let you bring a compiled executable, so everything is .ps1 and .psm1 text files run by the powershell.exe that is already in Windows, with a WinForms window. Its README says exactly that: built for environments that do not allow a compiled executable.
- **A distinction you confirmed:** the rule is 'no executable', not 'no compiled code', so inline C# through Add-Type is allowed when it helps. The default is still native PowerShell plus the stock WinForms/.NET pieces, for simplicity.

Where it lives: D:\Build Projects\Old PowerShell\PsSupportTool\ (the original app is in the SupportToolApp folder beside it).

3.2 What is in it

Module (folder under Modules\)	Tools	What they do
Pspt.Core	5 helpers	Shared plumbing, not a tool category: running a tool locally or on a remote computer, credentials, turning results into a table for the window.
Pspt.System	31	Services, printers, patches, processes, disk space, scheduled tasks, activation, temp cleanup, WSUS, Windows product key, auto-logon, keep-alive...
Pspt.Users	18	Local users and groups, RDP sessions, account lockout, password expiry, run-as...
Pspt.Network	15	Adapter info, DNS, WiFi, ping, traceroute, port test, NIC reset, DHCP statistics...
Pspt.Remote	26	Reboot, enable RDP, event log, shares, registry, gpupdate, hosts file, WinRM, Wake-on-LAN, run a script remotely, browse a remote folder, launchers for RDP / Computer Management / Event Viewer.
Pspt.Deploy	14	winget installs, registry push, update staging, file-drop-and-run, trusted sites, quick access...
Pspt.Ssh	4	An SSH session launcher (aliased 'ash', your choice: it sounds better) plus key-based run-a-command and copy-a-file, for PLCs and production equipment that do not speak WinRM.
Pspt.ActiveDirectory	9	Help-desk AD on your own domain: view a user, view and manage group membership, unlock an account, reset a password.
Pspt.M365	12	Help-desk Microsoft 365 through the Microsoft Graph PowerShell SDK: view a user, groups and Teams membership, unblock sign-in, reset a password.
Pspt.Exchange	19	Mailbox delegate / Full Access, Send As, forwarding, out-of-office, calendar delegation, distribution-group membership (Exchange Online module).
Gui\MainForm.ps1	-	The window: 355 lines. Launch-PsSupportTool.ps1 (29 lines) is the entry point.

Totals, by an actual count tonight: 148 tool functions in nine tool modules plus the core, in about 3,800 lines. PowerShell's parser read all 12 script files with 0 errors. (My earlier notes said 147; the difference is one function that moved from the Users module to the Remote count while the AD password reset was relocated.)

3.3 How it works (the part to study)

- **One function per tool, one pattern:** each tool is an advanced function that takes -ComputerName and -Credential when it can act on another computer, and is exported from its module. The pattern is the convention, and you told me you may drop your own device scripts straight into a module folder (for example Pspt.Ssh) and follow it yourself.
- **The window builds itself by reflection:** the launcher imports every Pspt.* module except Core, and MainForm.ps1 builds the tool tree from Get-Command -Module. Each parameter becomes a control read from the function's own metadata: a switch or bool becomes a checkbox, a ValidateSet becomes a dropdown, an array becomes comma-separated text, and anything else is a text box. So adding a tool means writing and exporting a function; there is no window code to touch.
- **Local or remote, same code:** Invoke-PsptTarget (in Pspt.Core) runs the scriptblock on this computer or through Invoke-Command -ComputerName, and returns real PowerShell objects either way. The MAUI app had to turn results into JSON because C# had to cross a process boundary; here there is no round trip.
- **Safety on anything that changes something:** those functions use [CmdletBinding(SupportsShouldProcess)] and $PSCmdlet.ShouldProcess(...), which gives -WhatIf and -Confirm for free, and the window shows a confirmation dialog first.
- **Credentials once, in the top bar:** target computer, domain, username, password. They are applied to every tool that has -ComputerName or -Credential. Leave the username and password blank to run as the identity you are already signed in as.

3.4 Decisions that shaped it

- Scope: you first said 'full scope, as many tools as can smoothly transition', then deferred to the smaller recommendation: the core toolset first (System, Users, Network, Remote, Deploy), extended later the same night.
- Added afterwards (September 19): SSH for the production equipment (SSH is separately approved at the new job, as a first-priority backup path), on-premises AD, Microsoft 365 and Exchange.
- M365 and Exchange are two separate modules on purpose: they are two logins with two different permission systems (an Entra ID role such as Helpdesk Administrator, versus a separate Exchange role group), and the mailbox actions you actually used (delegate, Send As, forwarding, out of office, calendar) are Exchange concepts that Graph does not cover.
- Deliberately left out: Email and Spam Response, Monitor, Reporting, Search, Sound Combine, Web Apps, and the Credentials and Settings pages from the MAUI app. And, in the M365 and Exchange modules: creating identities, licensing, and bulk or organisation-wide actions (message trace, compliance search, mass sign-out).
- The window layout was adjusted on the 19th: the tool-tree pane was too wide, so the splitter went from 300 to 220 and the credentials bar was split into two rows.

3.5 What a few tools need on top of Windows

Tool or module	Needs
Get-PsptDhcpStat (Network)	the DhcpServer RSAT module
Reset-PsptAdPassword (Active Directory)	the ActiveDirectory RSAT module (it uses that module, not raw ADSI, to match the source app)
Pspt.Ssh	ssh.exe and scp.exe on the PATH (Windows' built-in OpenSSH Client). Run-a-

	command and copy need key-based sign-in, because ssh.exe cannot take a password non-interactively; the session launcher opens a real terminal so password-only devices still work.
Pspt.M365	the Microsoft Graph PowerShell SDK (Install-Module Microsoft.Graph -Scope CurrentUser) and an Entra ID role granted to your account. That role is the real limit on what you can do, not this code.
Pspt.Exchange	the ExchangeOnlineManagement module and a separate Exchange RBAC role (a custom Help Desk role group). Different login and different permission grant from M365.

3.6 A caution about your old scripts

The older Azure2\ folder and the root-level *Form.ps1 scripts (and, by the same token, the phish-purge and mailbox tools under Azure2\Email\) were written for a different employer. You flagged that any tenant details hard-coded in them would be wrong, and the rule for the port is: treat them as a reference for what operations existed, never copy a tenant ID, domain, address or mailbox out of them, and rebuild each tool generic and parameterized. Technically they also cannot connect as they are: they use Exchange Basic-Auth remote PowerShell and the MSOnline and AzureAD modules, which Microsoft retired between 2022 and 2024, so they need a new sign-in layer (Connect-ExchangeOnline and Microsoft Graph), not just new values.

3.7 Status, and what is not done

- **Checked:** every function was parse-checked and smoke-tested as it was built, and the whole window was launched and looked at with all modules loaded (my notes from the sessions). Tonight I re-ran the parser over all 12 files: 0 errors.
- **Not committed to git:** git shows the PsSupportTool folder as untracked in the Old PowerShell repository (my notes say to ask you before committing). The last commits in that repo are from July 4, for the MAUI app.
- **No automatic tests yet:** there are no Pester tests. Compare the email downloader, which has self-tests in three languages.
- **No packaging step:** you run it from the folder with .\Launch-PsSupportTool.ps1.
- **The window runs each tool synchronously:** it can freeze briefly on a slow remote call. Fine for a first version; a background runspace (as in the email downloader's PowerShell window) is the known improvement.
- **Idea on the list:** config-driven quick-launch buttons for internal web tools (Altiris, ServiceNow), which open a browser.
- **Before using it at work:** M365 and Exchange only work with the modules installed and the roles granted to you; and, as with any script, your machine's execution policy is an IT policy question.

3.8 How to run it and study it

```
cd "D:\Build Projects\Old PowerShell\PsSupportTool"
.\Launch-PsSupportTool.ps1
Get-Command -Module Pspt.* | Group-Object Source | Select Name, Count      (after importing
the modules)
```

- Read Pspt.Core.psm1 first (142 lines, 5 helpers): Invoke-PsptTarget is the heart of it.

- Then read the smallest tool module, Pspt.Ssh.psm1 (90 lines), to see the one-function-per-tool pattern with ShouldProcess.
- Then Gui\MainForm.ps1 (355 lines) to see how Get-Command metadata turns into checkboxes, dropdowns and text boxes.
- Exercise: write one tiny new tool function in Pspt.System following the pattern, export it, launch the window, and find it in the tree without touching MainForm.ps1.

4. Worship Slides: the rebuild with hymn search

The problem: you remembered adding a hymn search earlier in the week but could not find the build. The source (worship_slides.py, worship_core.py) had it, dated September 13; the only exes were from July 1, so no build contained it.

- **What the feature is:** a 'Hymn Lookup' tab: pick a hymn from the local database (hymns.json, 224 hymns), preview it, Add as Slides. 'Scan Folder...' rebuilds the database from a folder of hymn .pptx files. 'Search Online (stretch)' is only a placeholder message. Separately, the parser inserts hymns automatically from cues in the bulletin (which is what saved you the time).
- **The rebuild:** archived dist\ into archive\2026-07-01-builds\ first (1,230 files, timestamps kept), then ran both PyInstaller specs (single-file 38 MB and folder version) with --clean -y.
- **A detail that matters:** the app looks for hymns.json next to the exe (it is not bundled), so it was copied beside both exes. Without it the tab says 'No hymn database found'.
- **A build lesson:** my first single-file build was 6 MB bigger because pygame (installed for Bordeom) got packed in. Adding pygame to the excludes list in both spec files fixed it.
- **Proof:** each exe was launched and its Hymn Lookup tab showed '224 hymn(s) loaded from hymns.json'. Then you ran a real packet: it parsed perfectly, found the 3 hymns referenced, and saved you at least 40 minutes.

Known quirk (not fixed): the worship lookup's Old Testament text comes from Sefaria, which mixes scholars' footnotes into the verses. See section 4.3.

Stale files: dist\ still holds WorshipSlides.zip and WorshipSlidesSingle.zip from July (without hymns). Make new zips before sharing.

5. Church Kids: the new children's lesson maker

4.1 What it is

A desktop window (kids_app.py, with all the logic in kids_core.py) that builds one list of slides from four sources, then makes a kid-friendly PowerPoint. It is a slimmed-down cousin of worship slides: no bulletin parsing, no hymns, no PDF reading.

Source	Details
Bible Stories	The 50 Open Bible Stories (unfoldingWord, CC BY-SA 4.0): Creation to Revelation, 7 to 18 short picture frames each. One frame becomes one picture-and-text slide. All 50 stories are a 0.1 MB download; pictures (16:9, stored at 1280x720) are fetched only for stories you add, then cached in library\ next to the program.

Bible Lookup	The worship-slides lookup (book, chapter:verse, Look up, Add as slide) with four reading levels: Standard = World English Bible; Simple = Free Bible Version (CC BY-SA); Simplest words = Bible in Basic English (public domain in the US); Very plain = Plain English Version (CC BY-SA, a partial Bible).
Word Document	A Heading 1 or 2 starts a slide; with no headings, blank lines separate slides (a short first line becomes the title); tables become one line per row.
Themes	Drawn by the program from PowerPoint shapes, so there is no artwork to license: Sunny Day, Rainbow, Starry Night, Garden, Ocean, Simple Cross light and dark, plus six plain colour themes.
Credits slide	Automatic, last slide, names every source used with its license; can be switched off.

4.2 How it was built and checked

- Spec first (SPEC.md), with seven decisions listed (each with a default) and your 'all your suggestions are go'.
- kids_core.py --selftest: 33 offline checks plus 9 more against the real websites (all 50 stories, a picture, every reading level, all 66 Bible book codes against the real API).
- tests\test_gui.py: 21 checks that drive the real window with a script (get stories, filter, add, look up, Word file, reorder, untick, generate).
- PowerPoint itself rendered the generated slides to images so the themes and text fitting could be looked at, not guessed.
- dist\ChurchKids.exe (24.5 MB) passes two built-in checks: --check (everything it needs is packed inside) and --check-net (downloads the 50 stories from inside the exe). Both exit 0.
- Your test deck: 40 slides, two whole stories plus Genesis 1:1-20 in Bible in Basic English, a correct credits slide.

4.3 Findings worth remembering

- **Sefaria's Old Testament is not clean:** Genesis 1:1 came back as 'When God began to createaWhen God began to create In contrast to others...' (JPS with footnotes mixed in), no verse numbers, GOD/ETERNAL for the divine name. Church Kids therefore defaults to bible-api.com (World English Bible); Sefaria stays available. The worship tool has the same quirk today.
- **The Plain English Version is partial:** 27 books (parts of Genesis, 1-2 Samuel, Psalms, Proverbs, Jonah, and 21 New Testament books) and not every chapter (Matthew 19 is missing). I first said 'New Testament only'; that was wrong.
- **No openly licensed true children's Bible exists;** the well-known ones are copyrighted. The open route is simple-English translations plus Open Bible Stories.
- **Open Bible Stories images:** the folder called 2160px is really 1920x1080. Real pictures average 234 KB each even at 1280 px, so a 28-picture deck is about 7 MB (I had estimated 2 MB per story). At 1024 px wide they would average about 127 KB with no visible loss on a 1080p projector: a possible tweak.
- **Licenses:** CC BY-SA means: credit the source and share your adaptation under the same license. That is why the credits slide is automatic and on by default.

ReadingLevelSamples.docx (in the ChurchKids folder) shows four passages at every level side by side, for the school director to choose a default. Sydney is reviewing tomorrow.

6. Bordeom: the fun collection

Your seventh-grade first program was a bouncing line; these are what a 1990s DOS toy looks like today. Each is one Python file with a headless --selftest. Requirements: numpy and pygame for warp, bounce and fractal; morse.py plays through Windows' winsound and uses numpy only to write WAV files.

Program	The idea	What to study in it
warp.py (15 checks)	An Acidwarp tribute. Draw ONE picture of palette index numbers 0-255 from maths (rings, spirals, waves, plasma, kaleidoscope, tunnel, a Julia fractal...), then rotate the colour palette every frame so colours flow through the shapes. Fade, new picture, repeat.	Index images and palette cycling; seamless palettes (cosine palettes whose ends meet); why only re-colouring, not redrawing, makes it fast.
bounce.py (12 checks)	A line (two endpoints) bouncing off all four walls with a fading trail. After a random wait, the next bounce is special: one line splits into 2-4; several lines kill the one that bounced.	reflect(): exact bouncing by 'unfolding' the path (a wall is a mirror), correct even for a step that crosses the screen twice; PCG64 random numbers seeded from OS entropy; the --rhythm option.
fractal.py (12 checks)	A Mandelbrot/Julia zoomer with colour cycling. Starts as a screen-saver 'dive' into spiral valleys; click to explore. A full 320x200 frame takes about 12 ms (it needed a maths coprocessor in the 1980s).	Escape-time iteration (z = z*z + c), dropping pixels as they escape, smooth colouring, why zoom stops near 1e-13 (float64 digits run out), how the dive targets were FOUND by computation after remembered coordinates went flat.
morse.py (19 checks)	Type text, hear Morse code; Go/Stop, speed, tone, repeat count, Farnsworth spacing, hide-and-reveal for learning by ear, Save as WAV (repeat N times or fill at least S seconds). Made for your father, a ham since 1956. MorseCode.exe (23 MB) for the ham club.	Standard timing (PARIS plus a word gap = 50 dits, which defines WPM); Farnsworth formula; scheduling each tone against the clock instead of adding delays; writing a WAV file with 5 ms fades to avoid clicks.

5.1 Your old BASIC randomizer, explained

INT(RND(1)*N)+1 gives a whole number from 1 to N. Nesting it (one draw sets the ceiling for the next) skews results toward small numbers. For N = 10 the values 1 to 10 come out about 29%, 19%, 14%, 11%, 8.5%, 6.5%, 5%, 3%, 2%, 1% (three levels deep: 51%, 21%, 12%, 7%...), instead of 10% each. That is a real technique (Knuth describes shuffling one generator with another). bounce.py --rhythm nested uses the continuous version (wait = min + range x u1 x u2; average 4.5 s against 6.0 s for flat); --rhythm deep is three deep (3.75 s). Press T while it runs to compare.

7. Where everything lives

What	Where
PowellTec Support Tool (original .NET app + pure PowerShell port)	D:\Build Projects\Old PowerShell\ (SupportToolApp = MAUI app, ToolsAvaloniaApp, PowellTecRelay, and PsSupportTool\ = the pure PowerShell port with Modules\Pspt.* and Gui\MainForm.ps1). Session records: C:\Users\ntadmin\.claude\projects\D--Build-Projects-Old-PowerShell\ (with memory\ notes)
Email downloader (repo, git)	F:\ClaudeProjects\email_downloader\ (SPEC.md, README.md with 'Where I left off', python\, powershell\, csharp\, testdata\)
Email downloader exe	...\email_downloader\dist\EmailDownloader\EmailDownloader.exe (zip for your boss: Desktop\EmailDownloader_for_boss.zip)

Test results (real email: keep private)	F:\ClaudeProjects\TestResults\ (outlook\python\junk99, outlook\sharp\t1 = C#, t2 = PowerShell, gmail\...)
Worship slides	F:\ClaudeProjects\AgentBuilds\Church\ (dist\WorshipSlides.exe and dist\WorshipSlides\, archive\2026-07-01-builds\, BuildNotesForSteve.txt, hymns.json)
Church Kids	F:\ClaudeProjects\ChurchKids\ (SPEC.md, README.md, kids_core.py, kids_app.py, tests\, ReadingLevelSamples.docx, dist\ChurchKids.exe)
Bordeom	F:\ClaudeProjects\Bordeom\ (warp.py, bounce.py, fractal.py, morse.py, README.md, dist\MorseCode.exe)
Workspace map (a fresh session reads it first)	F:\ClaudeProjects\CLAUDE.md
This guide	Google Drive: FromClaude\2026-09-19 Study Guide - Everything We Built.docx

8. Concepts worth studying (with where to see them)

Concept	What it means	Where to look
Spec first, tests second, code third	Write the rules (SPEC.md) and shared test data before the code, so several implementations can be proven identical.	email_downloader\SPEC.md and testdata\ ; ChurchKids\SPEC.md
OAuth 'public client' sign-in	A desktop app cannot keep a secret, so it opens the browser, you approve, and the browser returns to http://localhost with a code. Microsoft apps have no client secret here; Google's credentials.json 'installed' block is the equivalent.	python\outlook_downloader.py (msal); csharp\GmailProvider.cs
Microsoft sign-in authorities	/common (any account), /consumers (personal accounts only), /organizations (work accounts): the app registration's audience must match.	the AUTHORITY line in each Outlook provider
Gmail API vs Microsoft Graph	Two different mail APIs: Gmail returns raw messages with a query language (q); Graph exposes folders and $search. The tool hides the difference behind one Provider shape.	email_core.py and the two provider files
SPF, DKIM, DMARC	SPF: is this server allowed to send for this domain? DKIM: is the message signed? DMARC: do they line up with the From address? Mailing lists break alignment, so DMARC-only failures are not proof of forgery.	email_core.py: _AUTH_SPF, _SPF_RES, render_report sender drafts
Defanging links	Rewrite a URL so it cannot be clicked (hxxps, [.]) while staying readable and reversible.	SPEC section 12.1
Cross-language byte-for-byte testing	The same input must give exactly the same output in Python, PowerShell and C#. Differences found today: PowerShell 5.1 does not decode numeric HTML entities above U+FFFF; regex behaviour matches only if you avoid language-specific features.	testdata\expected*
Packaging: PyInstaller vs dotnet publish	PyInstaller bundles Python and your modules; it needs data files and hidden imports listed, and unused installed packages (pygame) can sneak in. dotnet publish --self-contained -p:PublishSingleFile=true bundles .NET. A windowed exe has no console, so build a --check flag whose result is the exit code.	*.spec files; kids_core.frozen_check; the dotnet publish command in section 2.8
Tk and threads	Background threads must not call Tk directly. Put results on a queue.Queue and let the window poll it. The test caught this bug.	ChurchKids\kids_app.py: _post, _poll, _run
Palette cycling	The picture stays fixed as numbers 0-255; only the colour table rotates. Cosine palettes are periodic, so they loop with no seam.	Bordeom\warp.py: pal_cosine, to_index
Reflection by unfolding; random number	Fold a position back into [0, L] with floor/modulo instead of looping bounces. PCG64 is a modern generator with an	Bordeom\bounce.py: reflect, Sim.next_period

generators	enormous cycle; old BASIC RND was weak, which is why nesting was a fix.	
Escape-time fractals	Iterate z = z*z + c; count steps until |z| passes a limit; smooth the count with log2(log2|z|); skip the cardioid and bulb; drop finished pixels.	Bordeom\fractal.py: compute
Morse timing and Farnsworth spacing	dit = 1.2/WPM seconds; dah = 3; gaps 1, 3, 7. Farnsworth keeps letters fast but stretches gaps: unit = (60/overall - 37.2/letter) / 19 seconds.	Bordeom\morse.py: timeline, spacing
Open licenses	Public domain (no conditions) vs CC BY-SA (credit, and share adaptations under the same license). Track the source of every text and picture; generate the credit automatically.	ChurchKids\kids_core.py: CREDIT_OBS, LEVELS, credits_text
Reflection-driven UI, and ShouldProcess	Instead of hand-building a control for every tool, the window asks PowerShell what each function's parameters are (Get-Command) and builds checkboxes, dropdowns and text boxes from that. Anything that changes state declares SupportsShouldProcess, which gives -WhatIf and -Confirm for free.	PsSupportTool\Gui\MainForm.ps1; any Set-/Reset- function in a Pspt.* module
Role-based access is the real boundary	In the M365 and Exchange modules the code cannot grant you power you do not have: the Entra ID role and the separate Exchange role group decide what works. Design tools as thin, help-desk-scoped helpers on top of that.	PsSupportTool README, 'Known external dependencies'
Windows PowerShell 5.1 and WinForms	A whole program can be one .ps1: a window built from System.Windows.Forms, network calls with .NET classes, a background runspace so the window does not freeze. Gotchas met today: ASCII-only files, $script: scoping, runspace parameter names, HtmlDecode above U+FFFF.	powershell\EmailDownloader.ps1: Show-Gui, the -Library switch
Treat a failing test as data	Several tests failed today because the TEST was wrong (wrong expected direction, wrong count, wrong threshold). Work out which side is wrong before changing anything.	see section 10

9. Command cheat sheet

Email downloader (run from F:\ClaudeProjects\email_downloader)

```
python python\email_downloader.py --selftest
powershell -NoProfile -File powershell\EmailDownloader.ps1 -SelfTest
dotnet run --project csharp\EmailDownloader -- --selftest
python python\email_downloader.py --cli --provider outlook --query "in:spam" --count 50 --
report --out F:\ClaudeProjects\TestResults\somewhere
python python\email_downloader.py --cli --provider gmail --query "from:someone" --count 20 --
html --out C:\path
python python\email_downloader.py                    (the window; PowerShell and C# have the
same window)
```

Worship slides (F:\ClaudeProjects\AgentBuilds\Church)

```
python -m PyInstaller worship_slides_onefile.spec --clean -y      (single file ->
dist\WorshipSlides.exe)
python -m PyInstaller worship_slides.spec --clean -y              (folder version)
copy hymns.json next to each exe afterwards; copy dist\ into archive\<date>-builds\ BEFORE
rebuilding
```

Church Kids (F:\ClaudeProjects\ChurchKids)

```
python kids_app.py
python kids_core.py --selftest            (add --live to also test the real websites)
python tests\test_gui.py
python make_reading_samples.py
python -m PyInstaller church_kids_onefile.spec --clean -y
dist\ChurchKids.exe --check              (exit code 0 = fine; --check-net also tests the
internet path)
```

Bordeom (F:\ClaudeProjects\Bordeom)

```
python warp.py        python bounce.py --rhythm nested        python fractal.py        python
morse.py
python morse.py "CQ CQ" --wpm 20 --fwpm 8 --loops 0        (Ctrl+C to stop)
python morse.py "73" --save 73.wav --seconds 120           (fill at least two minutes)
python warp.py --selftest   (and the same for the others)
```

10. Mistakes we made and what they teach

Kept here on purpose: you learn more from these than from the parts that worked the first time.

What went wrong	How it was caught	Lesson
I said the Plain English Version was 'New Testament only' (in the spec and to you).	The sample sheet showed Matthew 19 missing; its book list has parts of Genesis and Psalms.	A book count is not a coverage claim. Check the real data before stating a fact.
Sender drafts pointed at Google for a legitimate mailing-list message.	Your real inbox test (the church/Triad list).	Test on ordinary mail, not just on spam. Narrow a rule to the evidence that proves the thing.
I estimated 2 MB per story; real pictures make it about 3.7 MB.	Your real deck was 7 MB.	Estimates from synthetic data mislead; measure real data.
I said 8 sender drafts covered 14 messages; it is 13.	Re-adding the numbers for this guide.	Recount before repeating a number.
I said your README change was 'likely your own edit'; it was mine.	You said 'I didn't do that'; git history and file times showed my own commit.	Verify the record before attributing a change to anyone.
An exe rebuild silently did not happen (a running copy locked the file); the tests ran on the OLD file.	The size and pygame check did not change.	After a rebuild, check the file's timestamp before trusting the test.
pygame got packed into two exes.	Listing the contents of the exe (archive viewer); one check of mine was flawed too, because the file also records the exclude list.	Inspect what is really inside a package; make sure your check cannot pass by accident.
A scripted mouse click meant for a test window landed on your Gmail.	The screenshot showed Chrome, not the app.	Do not click at screen coordinates on a live desktop. Test through the program's own hooks and exit codes.
Threads calling Tk directly.	The scripted window test crashed with 'main thread is not in main loop'.	Use a queue and poll it. Automated tests find timing bugs a person rarely hits.
Several tests had wrong expected values (a bounce direction, a word's length in units, a frame-count threshold, the deck's slide count).	Working the case by hand.	Decide whether the code or the test is wrong before changing either.

A shell heredoc turned '\t' into real tab characters in a README.	Reading the line back.	Your own note in CLAUDE.md: write text containing backslashes with the Write/Edit tools, not shell heredocs.
Sefaria's Old Testament text has footnotes mixed in.	The reading-level sample sheet.	Look at real output from a data source before promising 'the exact same lookup'.
I left the PowellTec Support Tool port out of the first version of this guide, the biggest single project.	You told me: 'you missed the big one.'	I had only used this session's notes. Work done in a different project folder lives in that folder's own session records (and the D: drive), so search all of them before saying a summary is complete.
A search of the whole F: drive for one exe was far too slow.	You said 'that is not that big of a folder'.	Look in the folder where the thing should be first.

11. Open items and what to do next

Item	Notes
PowellTec PsSupportTool	Not committed to git (untracked). No Pester tests, no packaging step, the window runs tools synchronously. Ideas: quick-launch buttons for internal web tools; add your own device scripts to a module folder following the one-function-per-file pattern. M365 and Exchange need their modules and roles.
Church Kids feedback	Sydney reviews it tomorrow. The story text beside pictures is 20 to 24 pt: check it on a projector. Possible tweak: store pictures at 1024 px (about half the size).
Church Kids default reading level	The director reads ReadingLevelSamples.docx and picks one (the code defaults to Standard).
Church Kids phase 2	Project Gutenberg storybooks (Wee Ones' Bible Stories, Child's Story of the Bible, Bible Stories and Pictures, The Children's Bible); each needs a small hand-checked 'how to split into stories' rule.
Email downloader	Whois lookups for the sender IPs in a report (you do them; the tool never fetches). Open-source checklist is on hold until you say it is done. Optional: PowerShell and C# Gmail live tests were done; Outlook on all three done.
Worship slides	Make new zips (the ones in dist are July builds without hymns). Optional: make its Old Testament lookup use the cleaner source, as Church Kids does. 'Search Online' is still only a placeholder.
Bordeom	Which bounce rhythm should be the default (flat, nested, deep)? Ideas: a decoder for morse.py, random practice groups, a Windows screen-saver version.
Housekeeping	Delete dist\EmailDownloader\credentials.json.broken-2026-09-19-2255 when you are sure. Keep the Desktop zip as the only thing you send your boss.

12. A suggested way to study all this

- Start by running the tests: they are the fastest tour of what each thing does. Any project: its --selftest (or -SelfTest). A failing test is a to-do list.
- Read one SPEC.md all the way through (Church Kids' is the shortest, the email downloader's is the richest).
- Pick one concept from section 8, find the function named beside it, and change one number to see what breaks (for example: the score weight of a flag, the rhythm in bounce.py, the Farnsworth setting in morse.py).

- Read each README's 'Where I left off' first; it says what was done, what is untested and how to resume cold.
- When something surprises you, write it in the README's findings; that habit is what made today's mistakes useful.

Everything here is reproducible from the folders in section 7. Nothing in this guide contains a password, token or the contents of your credential files.